

Frontend Interview Preparation Plan (0–2 Years Experience)

Goal

Prepare:

- 50 Angular questions
- 50 HTML/CSS/JavaScript/TypeScript questions

Focus areas:

- Angular Components
- Services
- Directives
- Data Binding
- Forms
- Routing
- Angular CLI
- package.json
- HTML
- CSS
- JavaScript
- TypeScript

Recommended Preparation Plan (15 Days)

Day	Topic	Questions Target
Day 1	HTML Basics + Semantic Tags	10
Day 2	CSS Basics + Flex/Grid	10

Day	Topic	Questions Target
Day 3	JavaScript Basics	10
Day 4	Advanced JavaScript	10
Day 5	TypeScript Basics	10
Day 6	Angular Basics + CLI	10
Day 7	Components + Bindings	10
Day 8	Directives + Pipes	10
Day 9	Services + Dependency Injection	10
Day 10	RxJS + Observable + HttpClient	10
Day 11	Forms (Template + Reactive)	10
Day 12	Routing + Guards	10
Day 13	package.json + Build + Environment	10
Day 14	Mini Project Revision	Full Revision
Day 15	Mock Interview	Practice

Angular Interview Questions (50)

1. What is Angular?

Answer

Angular is a TypeScript-based frontend framework developed by Google for building dynamic single-page applications (SPA). It provides built-in features like components, routing, dependency injection, forms, HTTP communication, and state management support. Angular helps developers build scalable and maintainable enterprise applications.

2. What is SPA?

Answer

SPA stands for Single Page Application. In an SPA, the browser loads a single HTML page initially, and later updates content dynamically without refreshing the entire page. This improves user experience and provides faster navigation between pages.

3. What is a Component in Angular?

Answer

A component is the basic building block of an Angular application. Each component controls a part of the user interface and contains HTML, TypeScript logic, and CSS styles. Angular applications are created by combining multiple reusable components together.

Example:

```
@Component({  
  selector: 'app-home',  
  templateUrl: './home.component.html'  
})  
export class HomeComponent {}
```

4. What is a Service in Angular?

Answer

A service is a reusable TypeScript class used to handle business logic, API calls, shared data, and reusable operations. Services help keep components clean and improve code maintainability. They are usually injected into components using Dependency Injection.

5. Why do we use services?

Answer

We use services to separate business logic from UI logic. Services improve code reusability and help avoid duplicate code across components. They also make applications easier to test and maintain.

6. What is Dependency Injection?

Answer

Dependency Injection (DI) is a design pattern where Angular automatically provides required dependencies to classes. Instead of manually creating objects, Angular injects services when needed. DI improves modularity, testing, and loose coupling.

Example:

```
constructor(private todoService: TodoService) {}
```

7. What is Angular CLI?

Answer

Angular CLI is a command-line tool used to create, build, test, and manage Angular applications. It simplifies development by generating components, services, routing, and configurations automatically. It also provides commands for serving and building projects.

8. Common Angular CLI commands?

Answer

Angular CLI provides commands to automate Angular development tasks. Common commands include creating projects, generating components/services, serving applications, and building production bundles.

```
ng new my-app
ng serve
ng build
ng generate component home
ng generate service user
```

9. What is interpolation?

Answer

Interpolation is used to display component data inside HTML templates using double curly braces. Angular automatically updates the UI whenever the component value changes. It is commonly used for displaying text values.

Example:

```
{{ username }}
```

10. What is property binding?

Answer

Property binding is used to bind component data to HTML element properties. It allows dynamic updates of DOM properties like src, disabled, value, and class. Property binding uses square bracket syntax.

Example:

```
<img [src]="imageUrl" />
```

11. What is event binding?

Answer

Event binding is used to listen to DOM events such as click, input, submit, and keyup. When the event occurs, Angular executes a method from the component class. Event binding uses parentheses syntax.

Example:

```
<button (click)="save()">Save</button>
```

12. What is two-way binding?

Answer

Two-way binding combines property binding and event binding together. It keeps component data and UI synchronized automatically. Angular commonly uses ngModel for two-way binding.

Example:

```
<input [(ngModel)]="username" />
```

13. What are directives?

Answer

Directives are special Angular instructions that modify the behavior or appearance of DOM elements. Angular provides built-in directives as well as support for custom directives. Directives help create dynamic UI behavior.

Types:

- Component directives
 - Structural directives
 - Attribute directives
-

14. What is *ngIf?

Answer

*ngIf is a structural directive used to conditionally add or remove elements from the DOM. If the condition is true, the element is displayed; otherwise, it is removed. It helps improve performance by not rendering unnecessary elements.

Example:

```
<p *ngIf="isLoggedIn">Welcome</p>
```

15. What is *ngFor?

Answer

*ngFor is a structural directive used to iterate through arrays or collections. It dynamically creates HTML elements for each item in the collection. It is commonly used for displaying lists and tables.

Example:

```
<li *ngFor="let user of users">{{ user }}</li>
```

16. What is ngClass?

Answer

ngClass is an Angular directive used to dynamically add or remove CSS classes based on conditions. It helps apply styles dynamically depending on component state. It improves UI flexibility and readability.

Example:

```
<div [ngClass]="isActive ? 'active' : 'inactive'"></div>
```

17. What is a Pipe?

Answer

Pipes are used to transform data directly inside Angular templates. They help format values such as dates, currency, uppercase text, and percentages without modifying the original data. Angular provides both built-in and custom pipes.

Examples:

- uppercase
 - lowercase
 - date
 - currency
-

18. What is a custom pipe?

Answer

A custom pipe is a user-defined pipe used to transform data according to business requirements. Developers create custom pipes when built-in pipes are insufficient. Custom pipes improve code reusability and cleaner templates.

19. What is lifecycle hook?

Answer

Lifecycle hooks are special Angular methods triggered during different stages of a component's lifecycle. They allow developers to run custom logic during initialization, updates, and destruction. Lifecycle hooks are commonly used for API calls and cleanup.

Examples:

- `ngOnInit`
 - `ngOnDestroy`
 - `ngOnChanges`
-

20. What is `ngOnInit`?

Answer

`ngOnInit` is a lifecycle hook that runs after Angular initializes component data-bound properties. It is commonly used for API calls, initial data loading, and setup logic. It runs only once during component creation.

21. What is `ngOnDestroy`?

Answer

`ngOnDestroy` is a lifecycle hook that runs before a component is destroyed. It is mainly used for cleanup tasks such as unsubscribing Observables and clearing timers. Proper cleanup helps prevent memory leaks.

22. What is Observable?

Answer

Observable is an RxJS feature used to handle asynchronous data streams in Angular. Observables can emit multiple values over time and are widely used with HttpClient, routing, and events. They support operators like map, filter, and catchError.

23. What is subscribe()?

Answer

subscribe() is used to receive values emitted by an Observable in Angular and RxJS. Observables are lazy, so most Observables will not execute until subscribe() is called. It also allows handling success, error, and completion callbacks.

24. What is HttpClient?

Answer

HttpClient is an Angular service used for making HTTP requests such as GET, POST, PUT, and DELETE. It simplifies backend API communication and automatically works with Observables. HttpClient is commonly used inside Angular services for API integration.

Answer

HttpClient is an Angular service used for making HTTP requests such as GET, POST, PUT, and DELETE. It simplifies API communication and automatically works with Observables. HttpClient is commonly used in services.

25. How to call API in Angular?

Answer

In Angular, APIs are usually called using HttpClient inside a service class. The HttpClient methods return Observables, and components subscribe to them to receive data.

```
this.http.get('url').subscribe(data => {  
  console.log(data);  
});
```

26. What is ReactiveFormsModule?

Answer

ReactiveFormsModule is an Angular module that enables Reactive Forms functionality. It provides support for FormGroup, FormControl, FormBuilder, validators, and reactive form directives. It is commonly used for complex and scalable forms.

27. What is FormsModule?

Answer

FormsModule is an Angular module used for template-driven forms. It provides support for ngModel, two-way binding, and form directives. It is mainly used for simple forms.

28. What is FormControl?

Answer

FormControl represents a single input field in Angular Reactive Forms. It stores the field value, validation status, touched state, dirty state, and error information. Examples include username, password, and email fields.

29. What is FormGroup?

Answer

FormGroup represents a collection of multiple FormControl objects grouped together as a form. It tracks the overall form value and validation status. FormGroup is commonly used for login, registration, and profile forms.

30. What is FormBuilder?

Answer

FormBuilder is a helper service that simplifies the creation of Reactive Forms. Instead of manually creating FormGroup and FormControl objects, developers can use shorter and cleaner syntax. It improves readability and reduces boilerplate code.

31. What are validators?

Answer

Validators are used to validate form fields in Angular. They help ensure users enter valid data before form submission. Angular provides built-in validators such as required, minlength, maxlength, email, and pattern.

32. What is a custom validator?

Answer

A custom validator is a user-defined validation function created for special business rules. It returns null if the field is valid or an error object if invalid. Custom validators are commonly used for password strength and confirm password matching.

33. What is routing?

Answer

Routing allows navigation between different components/pages in Angular applications. Angular Router updates the displayed component without reloading the full page. Routing is essential for SPA applications.

34. What is routerLink?

Answer

routerLink is an Angular directive used for navigation between routes. It updates the browser URL and loads the corresponding component. It is preferred over normal anchor tags in Angular applications.

Example:

```
<a routerLink="/home">Home</a>
```

35. What is router-outlet?

Answer

router-outlet is a placeholder directive where Angular displays routed components. When navigation occurs, Angular loads the matched component inside router-outlet. It acts as the page rendering area.

36. What are route parameters?

Answer

Route parameters are dynamic values passed through URLs. They are used for loading data based on IDs or other values, such as user details or product information. Angular accesses them using `ActivatedRoute`.

Example:

`/user/:id`

37. What is lazy loading?

Answer

Lazy loading is a technique where Angular loads modules or components only when needed. This improves application startup performance and reduces initial bundle size. Large enterprise applications commonly use lazy loading.

38. What are route guards?

Answer

Route guards are Angular features used to control access to routes and navigation. They are commonly used for authentication, authorization, and preventing users from leaving pages with unsaved changes. Common guards include `CanActivate`, `CanDeactivate`, and `CanMatch`.

39. What is package.json?

Answer

package.json is an important configuration file in Angular and Node.js projects. It stores project metadata, npm scripts, dependencies, devDependencies, and package versions. Angular projects use this file to manage libraries and application configuration.

Answer

package.json is a configuration file that stores project metadata, dependencies, scripts, and package information. It helps manage npm packages and project configuration. Angular projects heavily depend on this file.

40. What is npm?

Answer

npm stands for Node Package Manager and is used to install and manage JavaScript packages. Angular applications use npm to install Angular libraries, third-party packages, and development tools. It also helps manage project scripts and package versions.

41. Difference between dependencies and devDependencies?

Answer

Dependencies are packages required for the application to run in production, such as Angular framework libraries. devDependencies are packages needed only during development, testing, or build processes, such as testing tools and TypeScript compilers. This separation helps optimize production builds.

Answer

Dependencies are packages required for the application to run in production. devDependencies are packages needed only during development, testing, or build processes. Examples include testing libraries and build tools.

42. What is angular.json?

Answer

angular.json is the main Angular workspace configuration file. It stores build settings, assets, styles, scripts, environment configurations, and project-level settings. Angular CLI reads this file during build and serve operations.

43. What is TypeScript?

Answer

TypeScript is a superset of JavaScript developed by Microsoft. It adds static typing, interfaces, classes, and advanced tooling support. Angular uses TypeScript because it improves maintainability and scalability.

44. Why Angular uses TypeScript?

Answer

Angular uses TypeScript because it provides type safety, object-oriented programming support, and better IDE tooling. TypeScript helps catch errors during development and improves maintainability in large applications.

45. What is interface in TypeScript?

Answer

An interface defines the structure of an object in TypeScript. It specifies the required properties and their types. Interfaces improve type safety and code readability.

Example:

```
interface User {  
  id: number;  
  name: string;  
}
```

46. What is enum?

Answer

Enum is a TypeScript feature used to define a collection of named constant values. It improves code readability and avoids hardcoded values in applications. Enums are commonly used for statuses, roles, and categories.

47. What is module in Angular?

Answer

A module is used to organize related functionality together in Angular applications. Traditionally Angular applications used NgModules to group components, services, directives, and pipes. Modern Angular also supports standalone components that reduce the need for NgModules.

Answer

A module is used to organize related functionality together in Angular applications. Traditionally Angular applications used NgModules to group components, services, and directives. Modern Angular also supports standalone components.

48. What is standalone component?

Answer

A standalone component is an Angular component that works without being declared inside an NgModule. It directly imports required modules and dependencies. Standalone components simplify Angular application structure.

49. Difference between constructor and ngOnInit?

Answer

The constructor is mainly used for dependency injection and basic object initialization. ngOnInit is used for component initialization logic such as API calls and data setup. ngOnInit runs after Angular initializes component properties.

50. What is change detection?

Answer

Change detection is Angular's mechanism for updating the UI when component data changes. Angular automatically checks for changes and updates the DOM accordingly. This keeps the UI synchronized with application state.
